

Prompt Maestro para Presentación: ORM y Sequelize

Copia y pega este prompt en un asistente de IA para estructurar y diseñar tu presentación técnica.

Actúa como un Diseñador de Presentaciones Técnicas y experto en Arquitectura de Software Backend. Quiero que crees el contenido de las diapositivas y las notas del orador (speaker notes) para una presentación técnica de 10-15 minutos enfocada exclusivamente en ORMs (Object-Relational Mapping), usando como caso práctico Sequelize con PostgreSQL en una aplicación Node.js.

NO enfoques la presentación en Express ni en rutas HTTP. La presentación debe girar en torno a:

1. Qué problema resuelve un ORM.
2. Cómo se define un modelo en código JavaScript y cómo se mapea a tablas reales.
3. Cómo se configuran las relaciones (Claves Foráneas) en Sequelize.
4. El flujo de trabajo real para alterar el esquema de base de datos de manera profesional usando Migraciones (up/down).

Estructura la respuesta diapositiva por diapositiva (en total 8-9 diapositivas). Para cada diapositiva provee:

- ****Título de la Diapositiva****
- ****Elementos Visuales Sugeridos**** (diagramas, código simplificado, palabras clave)
- ****Contenido/Viñetas**** (máximo 3-4 viñetas breves y concisas)
- ****Notas del Orador**** (lo que debo decir al exponer, en tono profesional, claro y explicativo).

Usa la siguiente estructura paso a paso basada en el código real del proyecto:

Diapositiva 1: Portada (ORM & Sequelize: Mapeo, Relaciones y Control de Versiones de Base de Datos).

Diapositiva 2: ¿Qué es un ORM y por qué usarlo? (La brecha entre objetos JS y tablas relacionales PostgreSQL. El principio DRY y la seguridad).

Diapositiva 3: Anatomía de un Modelo (Usa el ejemplo del modelo Bebida: explicar cómo se definen las columnas como 'nombre', 'precio' con sus restricciones en JS).

Diapositiva 4: Relaciones y Claves Foráneas en el ORM (Usa el archivo index.js: explicar cómo se conectan modelos como Venta y DetalleVenta usando hasMany y belongsTo).

Diapositiva 5: El Concepto de Migración (Por qué NO debemos alterar la base de datos manualmente. Las migraciones como el 'Git' o control de versiones de nuestro esquema de datos).

Diapositiva 6: Caso Práctico - Agregar una Columna (Paso 1 y 2): Generar la migración desde la CLI (npx sequelize-cli migration:generate) y editar el archivo con queryInterface.addColumn (método up) y queryInterface.removeColumn (método down). Explicar por qué es vital defaultValue cuando hay datos previos.

Diapositiva 7: Caso Práctico (Paso 3 y 4): Ejecutar la migración con 'npx sequelize-cli db:migrate' y actualizar el mapeo del modelo en JavaScript para que Sequelize reconozca la nueva propiedad.

Diapositiva 8: Rollbacks: ¿Qué pasa si cometemos un error? (Paso 5): Explicar el comando 'db:migrate:undo' y cómo ejecuta el método down() para devolver la base de datos a su estado anterior.

Diapositiva 9: Conclusiones y Buenas Prácticas (La inmutabilidad de las migraciones ejecutadas, coherencia de restricciones entre JS y SQL, y siempre programar el método down).

Mantén un tono académico pero práctico, ideal para desarrolladores que ya programan pero quieren dominar el flujo de base de datos con un ORM de manera profesional.